

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION WITH OPENCV

COURSE CODE: DSA0204

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry. (BTL 6)*

Assessment Tool Weightage: 40%

Code of Conduct:

I __Chalamala Sai Harshitha (192424346)__ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C. Sai Harshitha

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints:

- A. Limited processing power (mobile CPU)
- B. Real-time performance
- C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

Problem Definition

A mobile application must detect three facial expressions: happy, sad, and neutral. The system should use OpenCV for face detection and expression analysis while satisfying three constraints: limited mobile CPU processing power, real-time performance, and robustness to slight head movement and lighting changes.

Constraint Analysis

Constraint	Design Approach
Limited CPU	Use resized frames, lightweight OpenCV operations, and process only the face region.
Real-time performance	Minimize processing time and unnecessary operations.
Slight head movement	Continuously detect and update the face position.
Lighting changes	Use grayscale conversion and basic brightness/contrast normalization.

1. Frame Acquisition

The mobile camera continuously captures video frames using OpenCV. Each frame is passed to the processing pipeline for expression recognition.

2. Preprocessing

The captured frame is resized to a smaller resolution to reduce CPU usage. It is then converted to grayscale. Basic brightness and contrast normalization can be applied to reduce the effect of lighting changes.

3. Face Detection

An OpenCV **Haar Cascade classifier** can be used to detect the face in the frame. The detector identifies the location of the face so that further processing can be restricted to the relevant region.

4. Face ROI Extraction

The detected face is extracted as a **Region of Interest (ROI)**. Processing only the face region reduces computational requirements compared with processing the complete camera frame.

5. Feature Extraction and Classification

Facial characteristics around the **eyes, eyebrows, cheeks, and mouth** are analyzed to identify expression-related patterns. A lightweight classifier then categorizes the expression into one of three classes:

- **Happy**
- **Sad**
- **Neutral**

6. Handling Movement and Lighting

The face detection stage is continuously updated so that slight changes in face position can be handled. Grayscale conversion and brightness/contrast normalization help reduce the effect of moderate lighting variations.

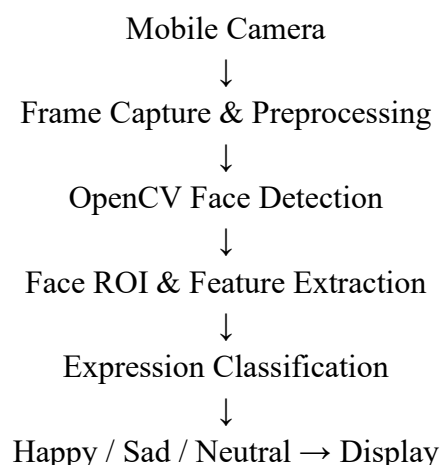
7. Result Display

The recognized expression is displayed on the mobile screen. For example, the output may show “**Happy**” when a happy expression is detected.

Constraint Satisfaction

- **Limited CPU:** Resizing, grayscale processing, ROI extraction.
- **Real-time performance:** Only essential processing is performed on each frame.
- **Head movement:** Continuous face detection updates the face location.
- **Lighting changes:** Normalization reduces the influence of illumination variations.
- **Three expressions:** The classifier produces happy, sad, or neutral as the final output.

Final Pipeline



Conclusion

The proposed OpenCV pipeline provides a practical solution for real-time facial expression recognition on a mobile device. Resizing, lightweight processing, face ROI analysis, continuous face detection, and lighting normalization help satisfy the given constraints while maintaining a suitable balance between computational efficiency and recognition accuracy.

2. Real-Time Object Detection on Mobile Device

A mobile application must:

- A. Detect objects using the camera
- B. Run in real time
- C. Use limited CPU and memory

Design an OpenCV-based pipeline from image acquisition to detection and display. Justify how your design ensures efficiency under constraints.

Problem Definition

A mobile application must detect objects through the device camera and operate in **real time** while using limited CPU and memory. Therefore, the OpenCV pipeline must reduce computational complexity without affecting the basic detection functionality.

Constraint Analysis

Constraint	Design Approach
Camera input	Capture live frames using OpenCV.
Real-time operation	Use resized frames and efficient processing.
Limited CPU	Select a lightweight object detection method.
Limited memory	Use smaller images and avoid unnecessary processing.

1. Frame Acquisition

The mobile camera continuously captures video frames using OpenCV. Each frame is passed to the processing pipeline for object detection.

2. Frame Resizing and Preprocessing

The captured frame is resized to a smaller resolution. This reduces the number of pixels that must be processed and consequently lowers CPU and memory usage.

3. Object Detection

A **lightweight object detection model** is applied to the processed frame. The detector identifies the objects present in the scene and determines their locations. A lightweight approach is preferred because the application operates under mobile CPU and memory limitations.

4. Bounding Boxes and Labels

For every detected object, a bounding box is drawn around its location. The corresponding object class is displayed as a label.

5. Display

The processed frame containing the detected objects, bounding boxes, and labels is displayed on the mobile screen. The process is repeated continuously for subsequent camera frames.

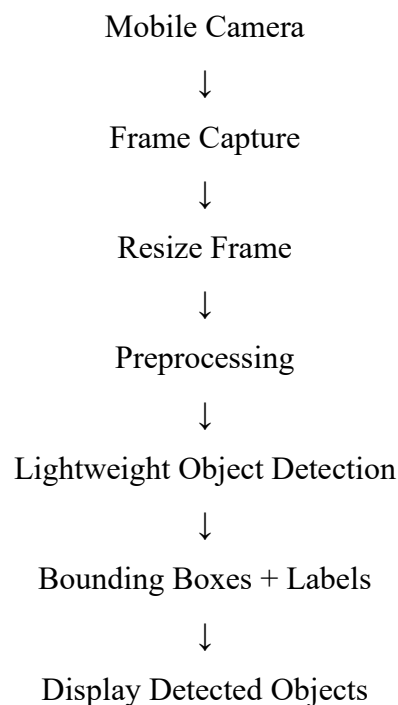
Efficiency Justification

The main strategy is to reduce the amount of data processed at each stage. Resizing the input frame reduces computational load, while a lightweight detector reduces CPU usage and memory requirements. Only essential preprocessing and detection operations are performed. Real-time performance is improved by avoiding unnecessary image-processing operations and maintaining a simple processing pipeline.

Constraint Satisfaction

- **Camera:** Live frames are acquired from the mobile camera.
- **Real-time:** Reduced-resolution frames and efficient detection minimize processing delay.
- **Limited CPU:** Lightweight processing reduces computational requirements.
- **Limited memory:** Smaller frames and an efficient detection model reduce memory usage.
- **Output:** Bounding boxes and labels clearly identify detected objects.

Final Pipeline



Conclusion

The proposed OpenCV pipeline provides a practical solution for real-time object detection on a mobile device. Frame resizing, lightweight detection, and minimal preprocessing reduce CPU and memory requirements while supporting real-time operation. Therefore, the approach satisfies the given mobile constraints while providing clear object detection results.

3. Smart Attendance System using Face Recognition

A system must:

- A. Capture faces from a webcam
- B. Recognize registered users
- C. Mark attendance automatically

Design a complete pipeline using OpenCV. Justify each stage and how the system avoids false recognition.

Problem Definition

A smart attendance system must use a webcam to capture faces, recognize registered users, and automatically mark attendance. The system should minimize false recognition by using appropriate face detection, feature comparison, and recognition thresholds.

Constraint Analysis

Requirement	Design Approach
Webcam input	Continuously capture frames using OpenCV.
Registered users	Store facial information of authorized users.
Automatic attendance	Mark attendance only after successful recognition.
Avoid false recognition	Use confidence/distance thresholds and repeated confirmation.

1. Face Capture

The webcam continuously captures video frames using OpenCV. Each frame is examined to identify faces present in the scene.

2. Face Detection

An OpenCV face detector such as a Haar Cascade classifier detects the location of faces in each frame. The detected face is extracted as a Region of Interest (ROI) for further processing.

3. Registered Face Database

Before using the attendance system, facial images of registered users are collected and associated with their identities.

4. Face Recognition

The detected face is processed and compared with the registered facial information. A recognition method determines the closest matching registered user.

5. False Recognition Prevention

The system should not mark attendance solely because a face produces the closest match. A recognition threshold is applied. If the similarity is insufficient or the distance exceeds the acceptable threshold, the face is treated as unknown.

6. Automatic Attendance

When a registered person is successfully recognized and passes the verification threshold, the system automatically records the person's identity and attendance.

7. Display Output

The recognized person's name can be displayed along with a message such as “**Attendance Marked**”. Unknown faces can be displayed as “**Unknown**” without recording attendance.

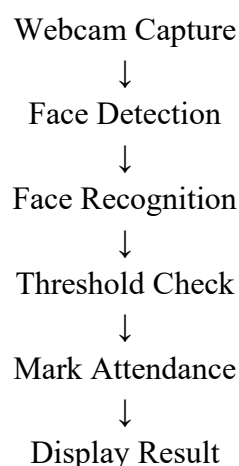
Justification of the Design

The pipeline separates **face detection** from **face recognition**, making the process systematic and efficient. Processing only the detected face ROI reduces unnecessary computation.

Constraint Satisfaction

- **Webcam capture:** OpenCV continuously acquires live frames.
- **Face detection:** Faces are located before recognition.
- **Registered-user recognition:** Captured faces are compared with stored facial information.
- **Automatic attendance:** Attendance is recorded after successful verification.
- **False recognition prevention:** Threshold checking and repeated confirmation improve reliability.

Final Pipeline



Conclusion

The proposed OpenCV-based smart attendance system provides an automated method for capturing, recognizing, and recording registered users. Face ROI processing improves efficiency, while recognition thresholds and repeated confirmation help reduce false recognition.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding ; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided